



École Nationale des Sciences Appliquées

Architecture des Microservices

TD n°5

Implémentation avec frameworks alternatifs

FI	GI 2
Elt Module	Microservice
AU	2025/2026

Réalisé par :
Wiam BenKrimen

Encadré par :
Pr. Zineb Besri

Table des matières

Introduction	2
1 Tableau comparatif	3
1.1 Performances et démarrage	3
1.2 Facilité de développement	3
2 Analyse critique	4
2.1 Quarkus – Points forts & points faibles	4
2.1.1 Points forts	4
2.1.2 Points faibles	4
2.2 Micronaut – Points forts & points faibles	4
2.2.1 Points forts	4
2.2.2 Points faibles	4
2.3 Recommandations par cas d’usage	4
Conclusion	6

Introduction

Objectif : Comparer les frameworks microservices Java alternatifs.

Frameworks étudiés :

- Quarkus (Supersonic Subatomic Java)
- Micronaut (JVM-based framework)

Critères de comparaison :

- Performance
- Temps de démarrage
- Facilité de développement

Livrables :

- Tableau comparatif
- Analyse critique

1 Tableau comparatif

1.1 Performances et démarrage

Critère	Quarkus	Micronaut
Temps de démarrage (mode dev)	0.5 - 1.5 s	1.5 - 3 s
Temps de démarrage (native)	0.01 - 0.05 s	0.05 - 0.1 s
Consommation mémoire (idle)	25 - 50 MB	40 - 70 MB
Throughput (re-quêtes/seconde)	45 000 - 55 000	40 000 - 50 000
Taille image native	15 - 25 MB	50 - 70 MB
Mode dev / hot reload	Intégré	Intégré

1.2 Facilité de développement

Critère	Quarkus	Micronaut
Courbe d'apprentissage	Modérée (Spring-like)	Modérée
Documentation	Excellente	Très bonne
Écosystème d'extensions	Très riche	Riche
Compatibilité Spring	Spring API compatible	Limitée
Outils (IDE)	Excellent	Bon
Live reload	Ultra rapide	Rapide

2 Analyse critique

2.1 Quarkus – Points forts & points faibles

2.1.1 Points forts

- Démarrage ultra-rapide (idéal pour serverless/FaaS)
- Bonne compatibilité Spring (migration facilitée)
- Compilation native GraalVM excellente
- Dev mode avec hot reload très réactif
- Très faible empreinte mémoire

2.1.2 Points faibles

- Compilation native plus lente
- Réflexion limitée (contrainte GraalVM)
- Moins mature que Spring sur certains aspects

2.2 Micronaut – Points forts & points faibles

2.2.1 Points forts

- Compilation AOT (Ahead-of-Time)
- Excellent pour les tests unitaires
- Bonne intégration avec GraalVM
- Support multi-langages (Java, Kotlin, Groovy)

2.2.2 Points faibles

- Taille image native plus grande que Quarkus
- Communauté plus petite que Quarkus/Spring
- Compatibilité Spring limitée

2.3 Recommandations par cas d'usage

Cas d'usage	Framework recom-mandé	Justification
Serverless / FaaS	Quarkus	Démarrage < 0.1s
Microservices conteneurisés	Les deux	Bonnes performances générales
Migration Spring existante	Quarkus	Spring API compatibility

Nouveau projet cloud	Micronaut	Compilation AOT
IoT / Edge computing	Quarkus	Très faible mémoire
Équipe Kotlin	Micronaut	Meilleur support Kotlin

Conclusion

Résumé des résultats :

Quarkus : Meilleur temps de démarrage, plus léger, meilleure compatibilité Spring

Micronaut : Bonnes performances générales, excellent AOT, bon pour Kotlin

Choix final selon le contexte :

- Pour une **nouvelle application cloud-native** : **Quarkus** (démarrage + mémoire)
- Pour une **équipe Kotlin** : **Micronaut**
- Pour une **migration depuis Spring** : **Quarkus**